What LLM Agent-Protocol Intrusion Detectors Actually Learn

Anonymous Submission

Abstract*—Large-language-model (LLM) agents now exchange tool descriptions, tool results and delegated tasks over the Model Context Protocol and the Agent-to-Agent protocol, and learned intrusion detectors are being proposed to flag malicious messages in this traffic. Whether such detectors learn attacks, or only the wording of benchmark attacks, is unknown. We build a measurement framework that renders five benchmarks as protocol episodes, splits them by server, case or template, and adds folds that hold out an entire attack wording or family. On identical episodes we compare a structured neural detector (selective state-space model, graph attention and chunk-level payload scoring) with an episode-length control, a TF–IDF lexical classifier, a label-free novelty score and a calibrated hybrid, over five seeds, under three alarm rules (validation-tuned, split-conformal and adaptive conformal) and with randomised-smoothing certificates. Chunk scoring adds up to 0.38 macro-F1 to the neural detector, yet the lexical classifier matches or beats it on every benchmark; the hybrid beats lexical in only 11 of 71 held-out runs; with a whole attack family held out (MCP-SafetyBench), AUROC falls to 0.60–0.67 for every model; and split-conformal thresholds give 17–19% false alarms against a 5% target when test servers are new, which adaptive conformal inference restores to 6–7%. For scientists, engineers and practitioners, the results show that current agent-security benchmarks reward template recognition, so detectors, and claims about them, must be judged on held-out attack families with drift-aware alarm control.

1. Introduction

Large-language-model (LLM) agents no longer act alone. They list and call tools through the Model Context Protocol (MCP), delegate tasks to other agents through the Agent-to-Agent protocol (A2A), and exchange the results as JSON-RPC messages. Each message is an attack surface: a tool description can carry hidden instructions (*tool poisoning*), a tool result can carry an injected command (*indirect prompt injection*), and an A2A task can reuse another client's context or redirect a webhook. A 2025 survey lists more than thirty attack techniques at the MCP layer alone [6], and MCPSecBench reports attack success above 85% on at least one commercial deployment [7].

A natural defence is the one networks already use: an *intrusion detector* that watches the traffic and raises an alarm. Learned detectors for agent traffic are appearing, and the newest benchmarks — MCPTox [10], MCP-SafetyBench [9], AgentDojo [14], InjecAgent [15] — make it easy to report high scores. Security machine learning has a history of high scores that do not survive contact with deployment: spurious correlations, sampling bias and inappropriate baselines are among the most common pitfalls [1], and detectors evaluated on random splits often collapse under time or family shift [2]. A concurrent study finds that simple tool-input and tool-output firewalls saturate four agent-security benchmarks and blames weak attacks and flawed metrics [3]. This paper asks what the learned detectors themselves pick up from such benchmarks.

We built a structured neural detector — a selective state-space model (SSM) over each agent's message stream, a Lipschitz-normalised graph-attention layer over the protocol graph, and a payload encoder that scores each tool description or sentence separately (*chunk scoring*) — and then measured, with controls, what it and three simple baselines learn. We call it SSM-GAT after its two main layers. Its one design choice that measurably helps, chunk scoring, is a contribution, but the main contribution is the measurement framework of Fig. 1 and what it reveals when SSM-GAT and simple baselines are put through it.

Research questions. **RQ1** (*detection*): in distribution, does a structured neural detector beat an episode-length control, a bag-of-words classifier and a label-free novelty score? **RQ2** (*mechanism*): which signal carries detection — the words, the payload encoding, or the protocol structure — and do architecture choices replicate across seeds? **RQ3** (*generalisation*): does any model detect attacks whose wording or family never appeared in training? **RQ4** (*operating points*): can the false-alarm rate be controlled, and are the models' probabilities trustworthy? **RQ5** (*guarantees and cost*): what do robustness certificates and latency measurements actually deliver?

Key findings.

- **F1 (RQ1).** The bag-of-words baseline matches or beats the neural detector on all five benchmarks (macro-F1 0.82–1.00 vs 0.50–0.96). A calibrated hybrid of detector, lexical and novelty scores differs significantly from lexical alone on one benchmark only, where it is *worse* (-0.040, Holm-adjusted $p = 0.02$).
- **F2 (RQ2).** The heaviest lexical features are the attack templates themselves. Chunk scoring is the one design choice that matters for the neural detector ($+0.20$ to $+0.38$ macro-F1 over one embedding

per message); the protocol graph and the SSM are not shown to help, and an ablation winner on two seeds (flat read-out, MCPTox macro-F1 0.84) did not replicate on five (0.50). Deleting the 50 most attack-indicative n-grams costs the lexical model 0.12 AUROC on MCPTox, and across 36 splits detection rises with the similarity of test attacks to training attacks (Spearman $\rho = 0.40$).

- **F3 (RQ3).** When attack wordings or families are held out, every model degrades; with an attack family held out on MCP-SafetyBench, AUROC is 0.60–0.67 for every model, close to chance (A2A's leave-one-class-out folds, at 0.47–0.61, cannot succeed by design). The hybrid improves on lexical in 11 of 71 held-out (fold, seed) runs.
- **F4 (RQ4).** Split-conformal thresholds, whose guarantee assumes calibration and test traffic are exchangeable, give 17–19% false alarms on MCPTox's unseen servers against a 5% target; adaptive conformal inference restores 6–7% at a recall cost. The neural detector is better calibrated than the lexical baseline before recalibration (mean ECE 0.08 vs 0.22).
- **F5 (RQ5).** Randomised smoothing certifies 0.36–0.98 accuracy at radius 0.05 in payload-feature space, but no certified radius exceeds 2.2σ and none transfers to text edits; a Lipschitz bound used in this detector family is unsound once residual paths are counted.

1.1. Contributions

- A **measurement framework** (Fig. 1, §3) for agent-protocol detectors: loaders that render five benchmarks as protocol episodes; grouped splits with a separate calibration half; held-out-template folds by wording, family and vulnerability class with a leakage audit; length, lexical and novelty controls on identical episodes; and a pre-registered analysis (hybrid weights, conformal and adaptive conformal alarms, paired bootstrap with Holm correction, sign test).
- A **five-seed study** (§4) answering RQ1–RQ5, including two negative results: the lexical baseline is not beaten, and an ablation winner did not replicate.
- **Chunk-level payload scoring** (§3.3), the one detector design choice that measurably helps, and a **template audit** (top features, feature removal, similarity of test attacks to training attacks) showing that episode-level scores on current corpora largely measure template recognition.
- A **certificate audit** (§4.5): measured smoothing curves, the ceiling on certifiable radii, and a counter-example showing that the block-product Lipschitz bound is not a bound.
- An **anonymised artifact** that reruns the whole GPU study in under three hours on one T4 and regenerates every table and figure.

2. Background, Threat Model and Notation

Agent protocols in one paragraph. In MCP, a *host* (the agent application) asks a *server* for its tools (`tools/list`); the reply is a list of tool names, natural-language descriptions and parameter schemas that the LLM reads to decide what to call. The host then calls a tool (`tools/call`) and passes the *tool result* back to the LLM. In A2A, an agent publishes an *Agent Card* describing its skills and sends tasks (`tasks/send`) to other agents, which stream status updates and artifacts back, optionally to a webhook. Every step is a JSON-RPC message, i.e. a small JSON object with a method name and parameters.

Threat model. A *trusted detector* sits between hosts, servers and agents and sees every message in clear text. The attacker controls content inside a message — a tool description, a tool result, an Agent Card field, a task parameter — and may know the detector's design and hyperparameters (grey box); for the certificates we also allow knowledge of the weights (white box). The attacker cannot forge signatures, compromise the detector's host or change its weights; supply-chain and memory attacks are out of scope. The defender wants (G1) few false alarms on benign traffic, (G2) high detection of attacks, (G3) robustness that can be stated as a guarantee, and (G4) throughput at the message rate of the deployment.

Measurement scope. We do not ask whether an attack *succeeds* against an LLM; the benchmarks already measure that. We ask whether a model that sees the traffic can tell attack episodes from benign ones, and whether that ability survives new attack wordings.

Notation. Table 1 collects every symbol. Vectors are bold only where a symbol is also used as a scalar; $\|\cdot\|_2$ is the Euclidean (straight-line) length of a vector; $[a, b]$ placed next to vectors means concatenation (stacking them into one longer vector); $\max_k$ applied to vectors takes the maximum of each coordinate separately; $\overline{e}$ is an average; $\mathbb{P}[\cdot]$ is a probability and $\mathbb{E}[\cdot]$ an expected (average) value.

3. Measurement Framework

Fig. 1 shows the six stages; this section describes each in turn.

3.1. Benchmarks and Episodes (stage ①)

None of the benchmarks records protocol traffic, so each case is rendered as an *episode*: the JSON-RPC messages an observer between host and servers would see around one user task, with a per-message label marking attacker-controlled content. Pinned source commits, construction rules and expected counts are in Appendix F.

MCPTox [10]: 1,312 tool-poisoning instances over 10 risk categories on 45 real MCP servers. Each attack episode lists the server's clean tools plus one poisoned tool; 533 benign episodes replay the same user queries with the clean tool list. **MCP-SafetyBench** [9]: 238 attack cases visible on the wire (poisoned or added tools, injected or forged results,

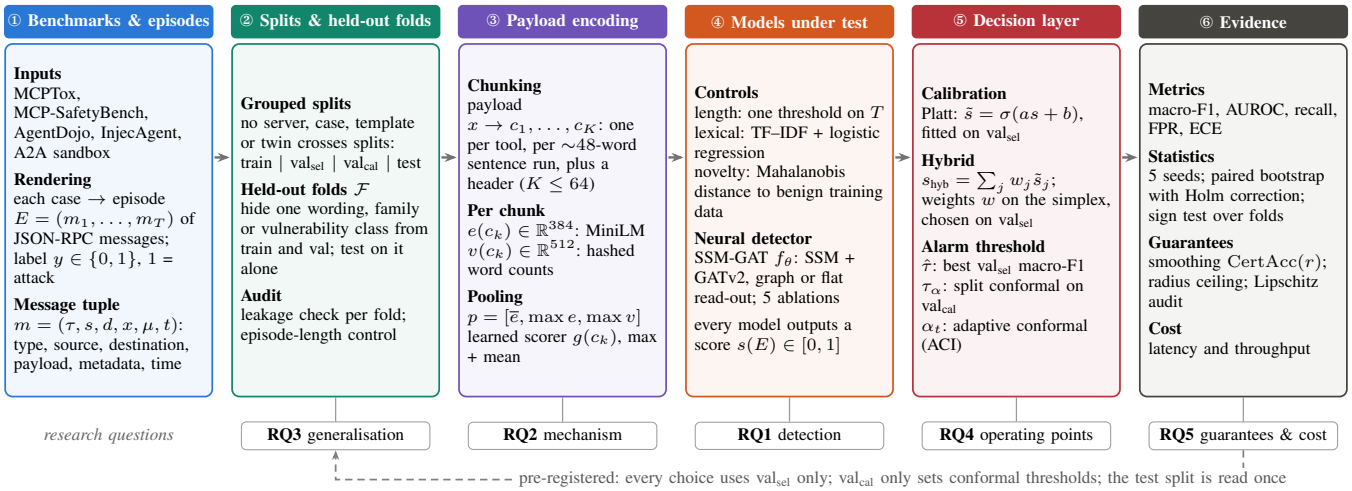


Figure 1. Measurement framework. Benchmarks are rendered as protocol episodes (①), split by group into training, a validation *selection* half, a validation *calibration* half and a test set, with additional folds that hold out one attack wording or family (②). Payloads are encoded per chunk (③) and scored by three controls and the neural detector on identical episodes (④). Scores are calibrated, optionally mixed, and turned into alarms by three threshold rules (⑤); the evidence layer (⑥) answers the five research questions. Symbols are defined in Table 1.

replayed calls, user requests) over 5 domains, plus 162 exact benign twins. **AgentDojo** [14]: the recorded agent runs shipped with the benchmark (no agent is re-run); a tool result is labelled as attacker-controlled when the injected string surfaces in it. **InjecAgent** [15]: 1,054 templated tool results with one injection slot; benign twins fill it with neutral text. **A2A sandbox**: synthetic A2A traces we generate for the 11 vulnerability classes of A2ABreak [8], whose artifact analyses the specification but contains no traffic; each attack episode has a benign twin with the same participants and task that differs only in the decisive field (requested scope, reused context identifier, webhook host, chunk index, ...). **MCPSecBench** [7] is loaded too, but its benign and attack episodes differ in the number of servers, so we use it only for message-level checks.

3.2. Splits, Held-Out Folds and Controls (stage ②)

Grouped splits. Episodes are split by *group* — MCPTox by server, MCP-SafetyBench by case (an attack and its benign twin together), A2A by attack/benign pair, AgentDojo by task suite and user task, InjecAgent by response template — and stratified by label, so that no server, case, template or twin appears in two splits. The validation split is halved again by a hash of the group into a *selection* half val_{sel}, used for early stopping, thresholds, calibration and hybrid weights, and a *calibration* half val_{cal}, used only for conformal thresholds (§3.5), so the latter is untouched by any fitted choice.

Held-out-template folds. Each fold $F \in \mathcal{F}$ removes one attack wording or family from training and validation and keeps only those attacks, plus the benign episodes of the test groups, in the test split. MCPTox: by wrapper style (`important_tag`, `plain`, `ignore_previous`) and separately by paradigm (Template-1/2/3). AgentDojo: by attack family (`important_instructions` and variants, `tool_knowledge`, `ignore_previous`, `injecagent`, `direct`, denial of service), using the recorded GPT-4o runs, the only ones attacked with many templates (capped at 100/25/40 attack episodes per family for train/val/test). InjecAgent: 4 hashed attacker-case groups. MCP-SafetyBench: by family (tool poisoning, name competition, result injection, host/user). A2A: leave one of the 11 vulnerability classes out. For every fold we verified that no held-out attack appears in training or validation and that no episode or group crosses splits.

Controls. The *length control* classifies by one threshold on T; its balanced test accuracy is 0.50 on four benchmarks and 0.53 on A2A, so a model must beat it to be doing more than counting messages. The *lexical baseline* is a logistic regression (a linear score passed through σ) on TF–IDF features of the episode's concatenated JSON text: each word and word pair is weighted by how often it appears in the episode, discounted by how many training episodes contain it (uni- and bigrams, minimum document frequency 2, at most 5×10^4 features, balanced class weights, regularisation $C = 1$). It has no sequence model, no graph and no protocol knowledge, so it measures how much words alone solve. The *novelty score* needs no attack examples: it averages an episode's message vectors p, projects them onto at most 32 principal components (the directions along which the training data vary most), and returns the Mahalanobis distance to the benign training episodes — the distance in units of the benign data's own spread, using a Ledoit–Wolf shrunk covariance [24], [25]. It is the only model that could, in principle, flag an attack family it has never seen.

TABLE 1. NOTATION, IN ORDER OF FIRST USE.

Symbol	Meaning
E, T	an episode (the messages around one user task) and its number of messages
m	one message (τ, s, d, x, μ, t): type τ (tool call, communication, capability, data, control), source s, destination d, payload text x, metadata μ (status, latency, auth), timestamp t
$y \in \{0, 1\}$	ground-truth label of an episode, 1 = contains attacker-controlled content
$\mathcal{F}$	set of held-out folds; each fold hides one attack wording, family or class
c_k, K	the k-th chunk of a payload and the number of chunks ($K \leq 64$)
$e(c)$, $v(c)$	384-dimensional MiniLM sentence embedding of chunk c; 512-dimensional hashed vector of its word and word-pair counts
$p \in \mathbb{R}^{1280}$	message vector $[\bar{e}, \max_k e(c_k), \max_k v(c_k)]$
g	learned chunk scorer (two-layer network, 896 inputs, 128 outputs)
f_θ, θ	neural detector and its trainable parameters
$s(E) \in [0, 1]$	a model's attack score for episode E
$\sigma(u)$	logistic function $1/(1+e^{-u})$, maps any number into $(0, 1)$
$\tilde{s}$	Platt-calibrated score $\sigma(a\, s + b)$, a, b fitted on validation
w_j	hybrid weight of model j; $w_j \geq 0$, $\sum_j w_j = 1$
$\hat{\tau}$, τ_α	alarm threshold tuned for macro-F1; split-conformal threshold
α, α_t	target false-alarm rate (0.05); ACI's working level at step t
γ	ACI step size (0.05)
σ (smoothing)	standard deviation of the Gaussian noise used for certification
r, $R(\cdot)$	a radius (size of a perturbation) and the certified radius of an input
Φ, Φ^{-1}	standard normal distribution function and its inverse
$\underline{p_A}$, n, α_{MC}	lower confidence bound on the top-class probability under noise, number of noise samples, and the chance that bound is wrong
L_f	Lipschitz constant: the most the output can change per unit change of input

3.3. Payload Encoding: Chunk Scoring (stage ③)

A single sentence embedding per message averages a poisoned tool description with every clean description in the same `tools/list` reply, and an injected sentence with the rest of a long tool result. We therefore split each payload into chunks $c_1, \ldots, c_K$: one per tool (name, description, parameter descriptions), one per run of whole sentences of about 48 words in long text fields, and one structural header (method, keys, short values), with $K \leq 64$. Each chunk receives a frozen `all-MiniLM-L6-v2` embedding $e(c_k) \in \mathbb{R}^{384}$ [20], [21] and a hashed vector $v(c_k) \in \mathbb{R}^{512}$ of logarithmic word and word-pair counts, both scaled to unit length. The message vector is

$$p = \left[\tfrac{1}{K} \textstyle\sum_k e(c_k),\ \max_k e(c_k),\ \max_k v(c_k) \right] \in \mathbb{R}^{1280}, \tag{1}$$

and the neural detector additionally passes every chunk through a learned scorer g and appends $[\max_k g(c_k),\ \ \frac{1}{K}\sum_k g(c_k)]$. This is *multiple-instance* pooling [22], [23]: a message is a bag of chunks, and one suspicious chunk can decide a coordinate instead of being averaged away. Maximum and mean change by at most as much as their inputs (they are 1-Lipschitz), so the pooling does not break the certificates of §3.7; the arg-max chunk also localises an alarm to a tool or sentence.

3.4. The Neural Detector Under Test (stage ④)

SSM-GAT reads a sliding window of messages. Each message becomes the vector $[p, \mu]$ plus the pooled chunk scores, projected to $d = 256$ dimensions. A *selective state-space model* [31], [32] then processes each agent's messages in order, keeping a running memory $h_t = \bar{A}_t h_{t-1} + \bar{B}_t u_t$ (the memory decays by a learned, input-dependent factor $\bar{A}_t$ and absorbs the new input u_t through $\bar{B}_t$), which costs time linear in the window length. A *graph attention* layer (GATv2 with LipschitzNorm [34], [36]) then lets each agent's state attend to the agents, tools and capabilities it exchanged messages with in the window — the *protocol graph*, whose nodes are agents, tools, capability descriptors and sessions and whose edges are typed, time-stamped messages. A *graph read-out* pools node states; a *flat read-out* skips the graph and pools the message sequence. A softmax layer outputs $s(E)$. Five ablations remove one part each: no graph attention, no SSM, flat read-out, a Transformer instead of the SSM, and no Lipschitz penalty. Training uses the AdamW optimiser, focal loss (cross-entropy that down-weights examples the model already classifies confidently), a penalty of weight $\lambda_L = 10^{-3}$ on the spectral norms of the weight matrices, class-balanced sampling and early stopping on val_{sel} macro-F1 (Appendix E). Its equations are in Appendix A; they are not needed to read the measurements.

3.5. Decision Layer (stage ⑤)

Calibration and hybrid. Each model's score is *Platt-calibrated* [26]: $\tilde{s} = \sigma(a\, s + b)$, with a, b fitted by logistic regression on val_{sel}, so that scores of different models live on one probability scale. The hybrid is $s_{\text{hyb}} = w_{\text{det}}\tilde{s}_{\text{det}} + w_{\text{lex}}\tilde{s}_{\text{lex}} + w_{\text{nov}}\tilde{s}_{\text{nov}}$ with weights on a 0.1 grid of the simplex ($w_j \geq 0$, $\sum_j w_j = 1$), chosen to maximise val_{sel} macro-F1 (ties: lower Brier score, the mean squared difference between score and label; then more balanced detector/lexical weights). Pairing a learned model with a novelty stage is a common intrusion-detection design: the novelty stage is meant to catch what the learned model has never seen.

Three alarm rules. (i) *Tuned*: $\hat{\tau}$ maximises macro-F1 on val_{sel} (the middle of the best range on ties). (ii) *Split conformal* [28]: with n_b benign calibration scores sorted in increasing order, τ_α is the $\lceil (n_b+1)(1-\alpha) \rceil$-th of them ($\lceil \cdot \rceil$ rounds up). If benign test traffic is *exchangeable* with benign calibration traffic (statistically indistinguishable in order), a benign test episode then raises an alarm with probability at most α, whatever the attacks look like. The rule needs $n_b \geq 19$ for $\alpha = 0.05$. (iii) *Adaptive conformal inference* (ACI) [29] drops exchangeability: it walks through the test

stream in random order, thresholds at the $(1-\alpha_t)$-quantile of the calibration benign scores, and, each time a benign label arrives, updates $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \mathrm{err}_t)$, where $\mathrm{err}_t = 1$ if that benign episode raised a false alarm and 0 otherwise. Too many false alarms lower α_t and raise the threshold; the long-run false-alarm rate converges to α for any sequence. ACI assumes benign labels arrive with some delay, as when an analyst triages alarms.

3.6. Evidence (stage ⑥)

Metrics. *Recall* is the fraction of attack episodes that raise an alarm; the *false-positive rate* (FPR) is the fraction of benign episodes that do. The F1 score of a class is the harmonic mean of its precision and recall; *macro-F1* averages the attack-class and benign-class F1, so a model cannot score well by labelling everything an attack. *AUROC* is threshold-free: the probability that a randomly chosen attack episode scores above a randomly chosen benign one (0.5 is chance, 1 is perfect). The *expected calibration error* [27] splits scores into $M = 10$ equal-width bins $\mathcal{B}_1, \ldots, \mathcal{B}_M$ and computes $\mathrm{ECE} = \sum_b \frac{|\mathcal{B}_b|}{N} \left|\mathrm{acc}(\mathcal{B}_b) - \mathrm{conf}(\mathcal{B}_b)\right|$, where N is the number of test episodes, conf is the mean score in a bin and acc the fraction of attacks in it; 0 means a score of 0.8 really is an attack 80% of the time.

Statistics. The main detector is trained with seeds 0–4; held-out folds with seeds 0–2 (A2A: seed 0); ablations with seeds 0–1. We report mean ± standard deviation over seeds. To compare the hybrid with the lexical baseline we use a *paired bootstrap*: resample test episodes 2,000 times with replacement, recompute both models' scores on the same resample, and read a 95% interval and a two-sided p-value from the differences; Holm's step-down correction [30] adjusts the five per-benchmark p-values for multiple testing. Over held-out folds we count the (fold, seed) runs in which the hybrid beats the lexical baseline and apply a two-sided sign (binomial) test. The analysis script was fixed before the five-seed run (it is `scripts/analyse_v3.py` in the artifact) and reads only saved scores.

3.7. Certificates (stage ⑥)

Randomised smoothing. Given the detector f and an input z (here the payload features of a window), the *smoothed* detector $\hat{f}(z)$ returns the class that f outputs most often when Gaussian noise of standard deviation σ is added to z [40]. From n noisy samples one computes a Clopper–Pearson lower bound (an exact binomial confidence bound) $\underline{p_A}$ on the probability of the top class, valid except with probability α_{MC}. Then no perturbation δ of length $\|\delta\|_2 \le R(z)$ can change $\hat{f}(z)$, where

$$R(z) = \sigma\, \Phi^{-1}\big(\underline{p_A}\big). \tag{2}$$

The *certified accuracy* $\mathrm{CertAcc}(r)$ is the fraction of test windows that $\hat{f}$ classifies correctly with $R(z) \ge r$. Because $\underline{p_A} \le \alpha_{MC}^{1/n}$, no radius above $\sigma\Phi^{-1}(\alpha_{MC}^{1/n})$ can be certified ($\approx 2.2\sigma$ at $n = 500$, $\alpha_{MC} = 10^{-3}$).

Lipschitz audit. A detector whose output can change by at most L_f per unit of input change (L_f is its Lipschitz constant) is certifiably stable within margin$/L_f$. For a stack of S residual blocks (each block adds its output to its input) after an input projection W_{in}, a sound bound *on the bounded operating domain of Assumption 1* (inputs in a ball, step sizes in an interval; Appendix B) is

$$L_f \le M_h\, L_{\text{gat}}\, \|W_{\text{in}}\|_2 \prod_{\ell=1}^{S} \big(1 + L_{\text{ssm}}^{(\ell)}\big), \tag{3}$$

where $L_{\text{ssm}}^{(\ell)}$ and L_{gat} bound the SSM blocks and the graph layer (Appendix B), M_h the output head, and $\|W\|_2$ is the spectral norm (largest stretch factor) of a matrix. The plain product $M_h L_{\text{gat}} \prod_\ell L_{\text{ssm}}^{(\ell)}$ used for this detector family omits the identity branch of every block; §4.5 shows it is not a bound. Our implementation clamps step sizes but does not clip inputs, so for the trained models (3) is a diagnostic, not a certificate.

4. Results

All runs used one NVIDIA T4 in fp32. The five-seed study (main runs, held-out folds, certificates and latency) took 2.8 GPU-hours; the earlier three-seed study that supplies the encoder and architecture ablations and the calibration measurement took 7.1. Exact outputs of both are in the artifact (Appendix I).

4.1. RQ1: Does the Neural Detector Beat Simple Controls?

Table 2 compares the four models on identical test episodes. On the **A2A sandbox**, where the attack is *structural* (a reused context identifier, an unverified webhook, an over-claimed Agent Card), the detector reaches macro-F1 0.962 ± 0.005 and AUROC 0.997, far above the 0.53 length control — and level with the lexical baseline (0.955, 0.995). On the four **payload** benchmarks the lexical baseline is ahead in macro-F1 everywhere: 0.890 vs 0.498 on MCPTox, 0.822 vs 0.651 on MCP-SafetyBench, 1.000 vs 0.882 on AgentDojo and 1.000 vs 0.929 on InjecAgent. The detector's seed variance is large where it is weak (± 0.080 to ± 0.135): on MCPTox its tuned thresholds are conservative (recall 0.34, FPR 0.03 on raw scores) although its ranking is useful (AUROC 0.78). The **novelty** score is near chance (AUROC 0.50–0.69) except on InjecAgent (0.92), whose benign episodes all share one filler sentence.

The **hybrid** puts almost all weight on the lexical score where lexical is strong (MCPTox 0.98) and most on the detector on A2A (0.66). Its macro-F1 is within 0.03 of the better single model on four benchmarks, and the paired bootstrap finds one significant difference from lexical alone: AgentDojo, where it is *worse* ($\Delta = -0.040$, 95% interval $[-0.077, -0.011]$, Holm $p = 0.020$). On MCP-SafetyBench the hybrid's AUROC (0.954) is the highest of any model, but the gain over lexical ($+0.020$, $[-0.010, +0.061]$) is not significant on 59 test episodes.

TABLE 2. RQ1, IN-DISTRIBUTION TEST RESULTS, MEAN OVER FIVE SEEDS (S.D. IN SMALL TYPE). EVERY MODEL SEES THE SAME TEST EPISODES; SCORES ARE PLATT-CALIBRATED ON VAL_{SEL} AND THRESHOLDED AT THE VAL_{SEL}-TUNED $\hat{\tau}$. DET.: SSM-GAT (FLAT READ-OUT, CHUNK SCORING). LEX.: TF–IDF LOGISTIC REGRESSION. NOV.: MAHALANOBIS NOVELTY (AUROC ONLY; MACRO-F1 0.45–0.63). HYB.: CALIBRATED HYBRID (WEIGHTS DET/LEX/NOV: A2A .66/.16/.18, MCPTOX .02/.98/0, MCP-SAFETYBENCH .44/.48/.08, AGENTDOJO .34/.66/0, INJECAGENT .48/.52/0). THE LENGTH CONTROL'S BALANCED ACCURACY IS 0.50 (0.53 ON A2A). n: TEST EPISODES (BENIGN). WITH 14 AND 3 BENIGN TEST EPISODES, MACRO-F1 ON AGENTDOJO AND INJECAGENT IS NOT INTERPRETED; COMPARE AUROC.

		Det.		Lex.		Nov.	Hyb.	
Benchmark	n **(ben.)**	**F1**	**AUC**	**F1**	**AUC**	**AUC**	**F1**	**AUC**
A2A sandbox	660 (330)	**.962**$\pm$.005	**.997**	.955	.995	.687	.960$\pm$.003	.996
MCPTox	265 (70)	.498$\pm$.080	.781	.890	**.917**	.498	**.891**$\pm$.002	.916
MCP-SafetyB.	59 (24)	.651$\pm$.135	.761	.822	.935	.626	**.846**$\pm$.064	**.954**
AgentDojo	104 (14)	.882$\pm$.053	.987	**1.00**	**1.00**	.523	.961$\pm$.054	**1.00**
InjecAgent	189 (3)	.929$\pm$.111	**1.00**	**1.00**	**1.00**	.923	**1.00**	**1.00**

Answer to RQ1. No: in distribution, a bag-of-words classifier is as good as or better than the structured detector on every benchmark, and combining them does not significantly help.

4.2. RQ2: What Signal Carries Detection?

The words are the attack templates. The heaviest attack features of the lexical model are the attack text itself: "important", "you must", "ignore the previous", "when using" on MCPTox; AgentDojo's fixed injection preamble ("before you can solve the task that I gave you"); the attacker's request on InjecAgent. We checked that no identifier leaks the label. Benign classes are small and narrow: InjecAgent's benign episodes share one filler sentence; MCP-SafetyBench's benign twins exist only for description-borne attacks and come mostly from two web-search servers (whose names are among the strongest *benign* features); and the A2A generator's attack templates have their own vocabulary ("jwks", "admin"), which likely explains the lexical baseline's 0.955 there. Episode-level scores on these corpora therefore largely measure recognition of a small set of templates.

Chunk scoring is the decisive encoder choice. Fig. 2 isolates the payload encoder (graph read-out, seeds 0–1, everything else fixed). One MiniLM embedding per message is at chance on MCPTox and MCP-SafetyBench (macro-F1 0.50, AUROC 0.50–0.57). Chunk mean+max pooling helps a little, the per-chunk word vector a little more, and learned chunk scoring most: macro-F1 0.70, 0.88 and 0.99 (AUROC 0.85, 0.97, 1.00). The attack occupies one chunk among several (5.5 chunks per message on average on MCPTox, 6.0 on MCP-SafetyBench), and only a pooling that one chunk can decide recovers it.

The protocol structure is not shown to help, and an ablation winner did not replicate. Table 4 removes one component at a time. On A2A every variant reaches

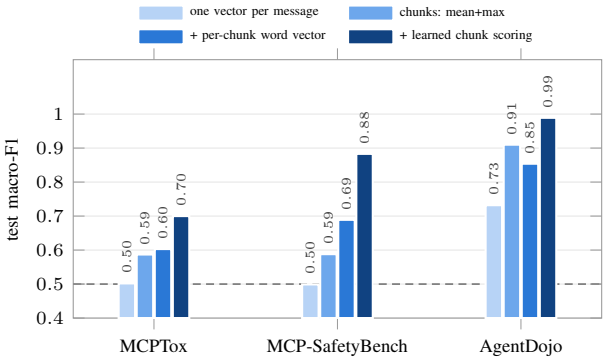


Figure 2. RQ2, payload-encoder ablation: test macro-F1, mean of seeds 0 and 1 (earlier three-seed study, graph read-out). Bars add, left to right, chunking, the per-chunk word vector and learned chunk scoring. Dashed line: chance (0.5). AUROC follows the same order (e.g. MCPTox 0.57 $\rightarrow$ 0.62 $\rightarrow$ 0.65 $\rightarrow$ 0.85).

macro-F1 0.952–0.962 and AUROC 0.996–0.997, within the seed spread. On MCPTox, in the earlier study, the full graph model was the *weakest* variant (0.731) and the flat read-out the strongest (0.841 ± 0.003, AUROC 0.945, two seeds). Because that choice was made on validation scores, we pre-registered the flat read-out as the main detector of the five-seed study. It did not replicate: with five seeds, and with early stopping and thresholds on the selection half of validation, the flat read-out reaches 0.498 ± 0.080 (AUROC 0.781) on MCPTox, while A2A is unchanged (0.962). We read this as a measurement result in its own right: on payload benchmarks with 70 benign test episodes the downstream architecture mostly adds variance, and two seeds are not enough to rank architectures.

A template audit. To test the template explanation directly, we ran three CPU measurements on the same splits (`scripts/template_audit.py`). The lexical AUROCs they recompute match the GPU study's to three decimals on all 36 splits, which confirms that the episodes are identical.

(i) What the words are. Table 3 lists the lexical model's heaviest features. On the attack side they are MCPTox's imperative wrapper words, AgentDojo's fixed preamble, InjecAgent's single attacker scenario and the A2A generator's field values; on the benign side, server and topic names ("serpapi", "sushi dinner", the channel "external_0") and A2A's completed-task status. MCP-SafetyBench's attack features are structural JSON-RPC keys ("tools call params") because its host/user and replay attacks sit in tool calls, which its benign twins do not contain.

(ii) How concentrated the signal is. We delete the k most attack-indicative n-grams of the full model and refit (Fig. 3c). On MCPTox, removing 10 costs 0.05 AUROC and removing 50 costs 0.12 ($0.917 \rightarrow 0.796$); on MCP-SafetyBench, 50 cost 0.05 and 1,000 cost 0.20. AgentDojo is untouched at $k = 50$, because its preamble is spread over many overlapping n-grams, and falls to 0.76 at $k = 1{,}000$. A2A hardly moves (0.944 at $k = 1{,}000$): its attacks change structured fields, whose values remain. InjecAgent stays at

TABLE 3. RQ2, TEMPLATE AUDIT (I): THE FIVE HEAVIEST ATTACK AND THREE HEAVIEST BENIGN FEATURES OF THE LEXICAL MODEL ON EACH BENCHMARK'S TRAINING SPLIT (WORD UNI- AND BIGRAMS AFTER TOKENISATION).

Benchmark	Attack features	Benign features
A2A	principal service; cdn example; https keys; example jwks; admin	done; task_state_completed message; example well
MCPTox	important; before; you must; when using; ignore	type string; string role; id
MCP-SB	args; system; enhanced; tools call; call params	serpapi fallback; return default; api primary
AgentDojo	that gave; can solve; gave you; task that; solve the	external_0; sushi dinner; discuss potential
InjecAgent	lock system; permanent access; friend amy; august smart; please grant	guests; permanent; thanks

TABLE 4. RQ2, ARCHITECTURE ABLATION: TEST MACRO-F1 / AUROC (CHUNK SCORING THROUGHOUT). ROWS 1–6: EARLIER STUDY, SEEDS 0–1 (FULL MODEL 0–2). LAST ROW: THE FLAT READ-OUT RE-RUN IN THE FIVE-SEED STUDY WITH THE SELECTION HALF OF VALIDATION. PARAMS: TRAINABLE PARAMETERS. L_f: RESIDUAL-AWARE LIPSCHITZ BOUND (3) OF THE SEED-0 A2A MODEL.

Variant	A2A sandbox	MCPTox	Params	L_f
Full (SSM + GATv2, graph)	.952±.002 / .996	.731±.095 / .850	4.10 M	5.4×10^6
SSM only (no GATv2)	.961±.015 / .997	.698±.137 / .834	2.35 M	3.6×10^5
GATv2 only (no SSM)	.954±.001 / .996	.795±.014 / .908	2.35 M	4.0×10^2
Flat read-out (no graph)	.960±.010 / .997	**.841**±.003 / **.945**	4.10 M	3.7×10^6
Transformer for SSM	.956±.000 / .996	.800±.116 / .935	4.46 M	∞
No Lipschitz penalty	.955±.001 / .996	.785±.027 / .898	4.10 M	3.6×10^8
Flat read-out, 5 seeds	.962±.005 / .997	.498±.080 / .781	4.10 M	—

1.00 even after 1,000 removals, because its benign class is one filler sentence and can be recognised from the benign side alone.

(iii) How familiar the test attacks are. For each test attack we isolate its *injected content* — the chunks of its attacker-controlled messages that occur in no benign episode (the poisoned tool, the injected sentence run, the altered A2A field) — and measure its TF–IDF cosine similarity to the nearest training attack (1 means the same wording was seen in training). In distribution the median is 0.96 on A2A (59% of test attacks have a near-duplicate with similarity ≥ 0.9), 0.84 on InjecAgent, 0.80 on AgentDojo and 0.75 on MCP-SafetyBench; the held-out folds lower it to 0.43, 0.32, 0.67 and 0.34. MCPTox is the exception: every poisoned description is unique (median 0.23 in and 0.20 out of distribution), and its detection rests on the few wrapper words of (ii). Across all 36 splits, AUROC rises with similarity (Fig. 3a,b; Spearman rank correlation $\rho = 0.40$, $p = 0.015$, for the detector and $\rho = 0.30$, $p = 0.07$, for the lexical baseline). The clearest outliers are InjecAgent's held-out folds, perfectly detected at similarity ≈ 0.33 through the benign-side shortcut of (ii).

Answer to RQ2. Words, and specifically the attack templates, carry detection on payload benchmarks: a few dozen n-grams carry most of MCPTox's signal, benign-side vocabulary alone separates InjecAgent, and detection is easiest where test attacks resemble training attacks. For the neural detector, how the payload is encoded matters far more than the sequence or graph model on top.

4.3. RQ3: Do Models Recognise Unseen Attack Templates?

Table 5 and Fig. 4 report the mean over held-out folds; per-fold results are in Appendix C. Fig. 4 draws each model's AUROC on the five benchmarks as a pentagon: a model that ranks perfectly touches the outer ring, and the dashed red ring is chance. In distribution (left) the lexical and hybrid pentagons fill the chart; held out (middle) every pentagon shrinks, and the right panel shows by how much.

Wording held out (MCPTox, AgentDojo). The lexical baseline loses little ranking quality (MCPTox AUROC 0.917 → 0.870, AgentDojo 1.00 → 0.98) but its tuned threshold no longer fits: on MCPTox its recall falls to 0.29 and 0.32 on the unseen `important_tag` and `plain` wrappers. The detector drops more (MCPTox AUROC 0.781 → 0.669; AgentDojo 0.987 → 0.915) and becomes conservative (MCPTox recall 0.24, FPR 0.08). AgentDojo's held-out folds use the recorded GPT-4o runs, the only ones attacked with many templates, while its main split uses Claude 3.5 Sonnet runs, so its change from in-distribution to held-out mixes model shift with template shift. The hardest AgentDojo family for both is `direct`, which states the attacker's goal without the wrapper text the other families share (detector 0.62, lexical 0.87 macro-F1). The detector beats lexical on two of six AgentDojo folds (`ignore_previous` 0.86 vs 0.80, `injecagent` 0.91 vs 0.90).

Family held out (MCP-SafetyBench, A2A). All models are near chance: AUROC 0.60–0.67 on MCP-SafetyBench and 0.47–0.61 on A2A. On A2A this is expected — each vulnerability class is a different protocol violation, and nothing in the other ten says what the eleventh looks like — but it is also the case where a structural detector was supposed to help. The novelty score, which needs no attack examples, does not rescue either benchmark (AUROC 0.62 and 0.48). *InjecAgent* stays saturated because unseen attacker cases still share the injection slot and the benign filler.

Does the hybrid help under shift? The hybrid beats the lexical baseline in macro-F1 in 11 of 71 held-out (fold, seed) runs and in AUROC in 9 of 71; ties, common on saturated folds, count as non-wins, so a sign test rejects "the hybrid wins half the time" ($p < 10^{-8}$). Only on MCP-SafetyBench does it win more often than not (8 of 12).

Answer to RQ3. No model generalises across attack *families*; across *wordings*, the lexical baseline keeps its ranking but not its threshold. Claims of generalisation on these benchmarks need family-level held-out evaluation.

4.4. RQ4: Can False Alarms Be Controlled, and Are Scores Trustworthy?

False alarms. Fig. 5(a) shows the test FPR at a 5% target. On **A2A**, where calibration and test benign episodes

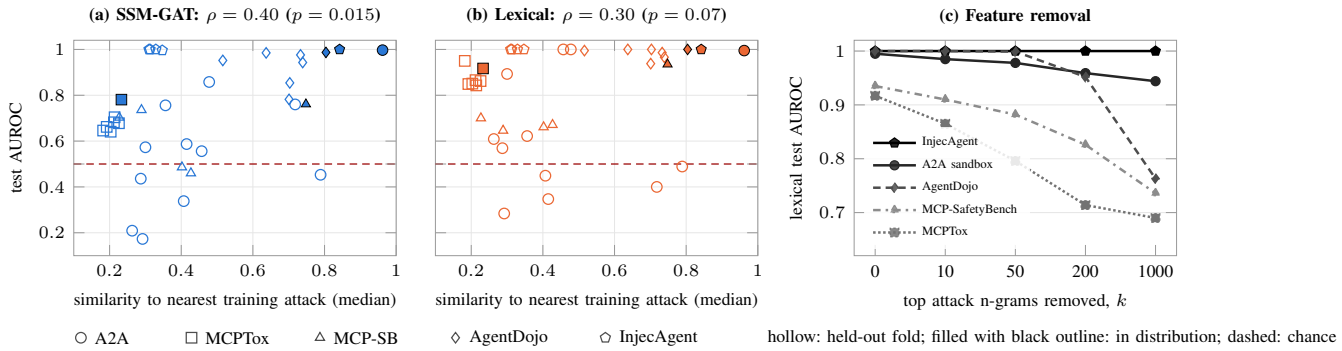


Figure 3. RQ2 and RQ3, template audit. (a, b) Test AUROC against the median TF–IDF similarity between each test attack's injected content and its nearest training attack, for all 36 splits (5 in distribution, 31 held-out folds); ρ is Spearman's rank correlation. Detection is easier when test attacks resemble training attacks; InjecAgent (pentagons) is detected perfectly at low similarity through its single benign filler. (c) Lexical AUROC after deleting the k most attack-indicative n-grams of the full model and refitting: a few dozen MCPTox wrapper n-grams carry most of its signal.

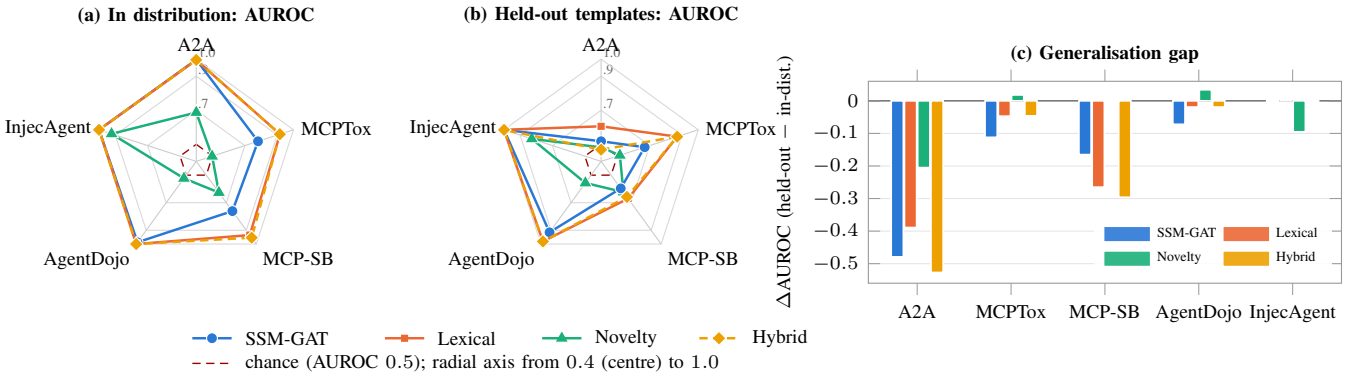


Figure 4. RQ1 and RQ3 at a glance. (a, b) AUROC of each model on the five benchmarks, in distribution and averaged over held-out folds; values are those of Tables 2 and 5. (c) The change from (a) to (b): the family-level benchmarks (A2A, MCP-SafetyBench) lose most, and the novelty score loses little only because it had little to lose.

TABLE 5. RQ3, HELD-OUT TEMPLATES: TEST MACRO-F1 AND AUROC, MEAN OVER FOLDS OF THE PER-FOLD SEED MEANS (SEEDS 0–2; A2A SEED 0); FOLDS IN PARENTHESES. FIG. 4(C) GIVES THE CHANGE FROM TABLE 2. WINS: (FOLD, SEED) RUNS IN WHICH THE HYBRID'S MACRO-F1 EXCEEDS THE LEXICAL BASELINE'S.

	Det.		Lex.		Nov.	Hyb.		
Held out (folds)	**F1**	**AUC**	**F1**	**AUC**	**AUC**	**F1**	**AUC**	**Wins**
A2A, class (11)	.374	.518	**.467**	**.606**	.482	.399	.469	2/11
MCPTox, wording (6)	.531	.669	**.675**	**.870**	.516	.661	**.870**	1/18
MCP-SB, family (4)	.464	.596	.528	**.670**	.623	**.568**	.658	8/12
AgentDojo, family (6)	.798	.915	**.902**	**.981**	.557	**.902**	**.981**	0/18
InjecAgent, case (4)	.924	.999	**.955**	**1.00**	.828	.946	.999	0/12

come from the same generator, split conformal does what it promises: FPR 0.033–0.048 for every model, with recall 0.96 for the detector, lexical baseline and hybrid. On **MCP-Tox** it does not: FPR 0.171 (detector), 0.186 (lexical) and 0.186 (hybrid), more than three times the target. The guarantee assumes exchangeable benign traffic, but the test split contains *servers* that calibration never saw, and benign tool lists from new servers score higher. ACI restores the target on MCPTox (0.057–0.071) by raising the threshold, and pays in recall (0.51–0.60 → 0.22–0.23). MCP-SafetyBench and AgentDojo have too few benign calibration episodes for split conformal ($n_b < 19$); there ACI keeps the detector at 0.058 and 0.114 FPR, but with 24 and 14 benign test episodes one false alarm moves the rate by 0.04–0.07. Under ACI, the hybrid gives the best recall on MCP-SafetyBench (0.69 at FPR 0.075) and the detector on AgentDojo (0.65 at 0.114).

Calibration. Fig. 5(b) reports ECE on the earlier study's recorded predictions (graph read-out, three seeds), the run whose per-episode scores we hold. Before recalibration the neural detector is much better calibrated than the lexical baseline on every benchmark (mean ECE 0.083 vs 0.219): the balanced, regularised logistic regression squeezes its probabilities toward 0.5 even when it ranks perfectly (AgentDojo: AUROC 1.00, ECE 0.29). Platt scaling on validation removes most of the difference (mean 0.043 vs 0.068), except on MCPTox, where unseen servers again break the

calibration (0.110 and 0.203 after recalibration).

Answer to RQ4. Partly. Distribution-free alarm control holds only when benign test traffic resembles benign calibration traffic; when servers change it fails silently, and an adaptive rule restores it at a large recall cost. Probabilities need recalibration, and recalibration is also limited by server shift.

4.5. RQ5: Certificate Audit and Cost

Smoothing certificates. Fig. 6 plots $\mathrm{CertAcc}(r)$ for the seed-0 model of each benchmark. Noise is added to the detector's whole payload input — the pooled vector p and every chunk feature — with 100 test windows (59 for MCP-SafetyBench), $n_0 = 32$ selection and $n = 500$ estimation samples, and $\alpha_{MC} = 10^{-3}$. At $\sigma = 0.05$ and $r = 0.05$, certified accuracy is 0.90 on A2A, 0.60 on MCPTox, 0.36 on MCP-SafetyBench, 0.93 on AgentDojo and 0.98 on InjecAgent. Larger noise buys radius only where the model is already confident: at $\sigma = 0.25$, $\mathrm{CertAcc}(0.25)$ is 0.69 on InjecAgent and 0.24 on AgentDojo but at most 0.03 elsewhere, with abstention (no class reaching the bound) up to 0.54. InjecAgent's curve is high partly because 98% of its windows are attacks. The radii are measured on unit-length embeddings; we have not measured how far a text edit moves an embedding, so none of them is a text-level guarantee.

Lipschitz audit. At an effective spectral cap of 0.5 on every linear layer, the block-product quantity is 0.023, but projected gradient ascent finds inputs whose output changes 0.083 per unit of input change, 3.6× more: the block product is not an upper bound. The residual-aware bound (3) is 3.09, which the probe respects (Appendix B, Table 7). On the trained benchmark models it is vacuous, 4×10^2 to 3.6×10^8 (Table 4); the Lipschitz penalty lowers it 66× on A2A without changing detection, and it is infinite for the Transformer variant, whose dot-product attention has no global Lipschitz constant [37]. The certificates above therefore rely on smoothing alone.

Latency. Table 6 separates the detector from the payload encoder. One detection window costs 5.4–6.1 ms on a T4, and batching brings the detector to 21,000–37,000 messages per second. The chunked encoder is the bottleneck: 1.3–3.5 ms per message in batches, i.e. about 290–760 messages per second end to end, and 8–19 ms for a single message (MCPTox tool lists have the most chunks). We did not measure an optimised or fused-kernel deployment.

Answer to RQ5. Smoothing gives non-trivial certified accuracy at small feature-space radii, but nothing certified here reaches text edits, and the Lipschitz route is unsound in its common form and vacuous in its sound form.

TABLE 6. RQ5, LATENCY ON ONE T4 (FIVE-SEED STUDY, FLAT READ-OUT). DETECTOR: MEDIAN OF 100 FORWARD PASSES AFTER 10 WARM-UPS ON PRE-COMPUTED PAYLOAD FEATURES; "WINDOW" IS ONE DETECTION WINDOW OF 3–8 MESSAGES; THROUGHPUT AT THE BATCH OF 1, 8, 32 OR 128 WINDOWS THAT MAXIMISES IT. ENCODER: CHUNKING, MINILM AND WORD HASHING PER MESSAGE, ONE AT A TIME AND IN BATCHES OF 200.

	Detector		Encoder (ms/msg)	
Benchmark	**ms/window**	**msg/s**	**one**	**batched**
A2A sandbox	6.14	23,312	8.15	1.92
MCPTox	5.41	33,371	18.61	2.01
MCP-SafetyBench	5.95	25,458	8.68	3.50
AgentDojo	5.45	21,486	8.72	2.20
InjecAgent	5.56	36,558	8.11	1.31

5. Lessons for Benchmark Builders and Practitioners

L1: Report a lexical baseline and a length control on identical splits. Both are cheap, and here they set the bar that a structured detector failed to clear. Pair them with a feature-removal curve and a similarity-to-training measure (Fig. 3); together they show in minutes whether a benchmark rewards templates. A detector paper that omits them cannot tell template recognition from attack understanding.

L2: Hold out wordings and families, not only episodes. Random or grouped splits keep every attack template in training. A held-out-family protocol, which our loaders provide for five benchmarks, is what separates the models' generalisation (Fig. 4c).

L3: Benchmarks need large, diverse benign classes. Three of five benchmarks have 3–24 benign test episodes; false-alarm rates on them move by 0.04–0.33 per episode, and their benign text is narrow enough to be a feature. Recorded benign traffic from many servers would help more than more attacks.

L4: Do not trust a false-alarm guarantee across servers. Split-conformal thresholds overshot their target threefold when test servers were new. Deployments should calibrate per server population or use an adaptive rule such as ACI, and should budget for the recall it costs.

L5: Encode payloads at the level where attacks live. One embedding per message hid single poisoned tools; per-chunk scoring recovered 0.20–0.38 macro-F1. This holds for any detector that reads tool lists or long tool results.

L6: Rank architectures with enough seeds. A two-seed ablation winner lost 0.34 macro-F1 under five seeds. On benchmarks this small, seed variance can exceed architecture effects.

6. Discussion and Threats to Validity

Construct validity. Episodes are reconstructed from benchmark cases, not captured traffic; the A2A sandbox is synthetic, with one generator template per vulnerability class, so its high in-distribution scores say more about the generator than about deployed A2A traffic. **Internal validity.** Every tuned choice used $\mathrm{val}_{\mathrm{sel}}$ and the analysis was fixed before the five-seed run; the main configuration was, however, chosen on the earlier study's validation scores, and that choice did not replicate (§4.2). The paired bootstrap, conformal and ACI results were computed on the compute

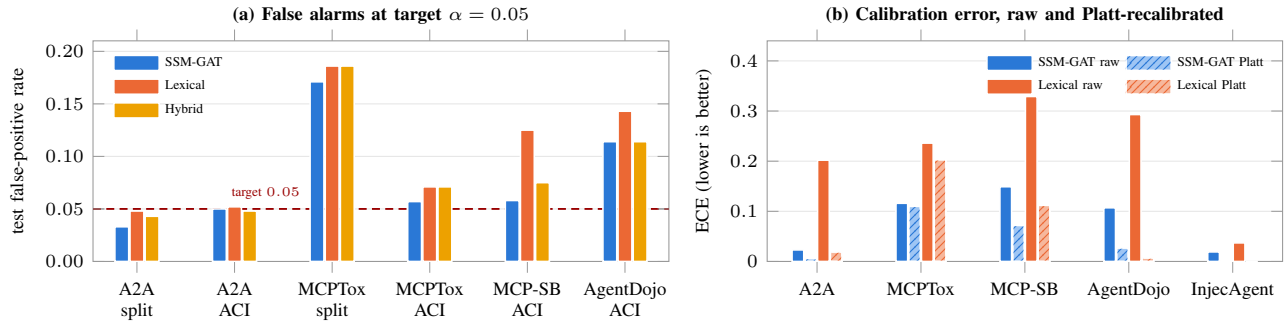


Figure 5. RQ4, operating-point reliability. (a) Test FPR under split conformal ("split", A2A and MCPTox only: the other benchmarks have fewer than 19 benign calibration episodes) and adaptive conformal inference ("ACI"); five-seed study, $\alpha = 0.05$. Split conformal overshoots on MCPTox, whose test servers are unseen; ACI brings every model back near the target. (b) Expected calibration error per benchmark, before (solid) and after (hatched) Platt recalibration on validation; earlier three-seed study, mean over seeds, $M = 10$ bins.

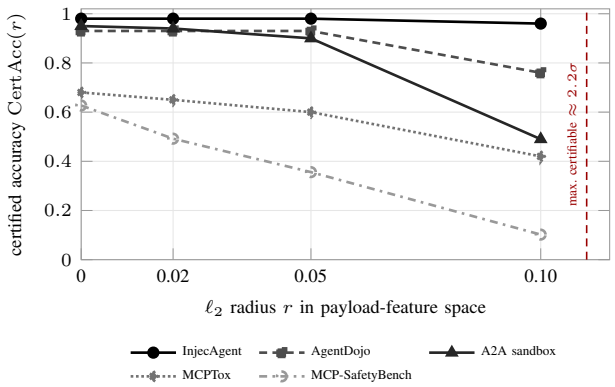


Figure 6. RQ5, measured certified accuracy versus radius (five-seed study, seed-0 models, $\sigma = 0.05$, $n = 500$, $\alpha_{MC} = 10^{-3}$). No radius beyond ≈ 0.11 can be certified at this σ and n.

platform from the five-seed run's saved per-episode scores; those files were lost when the platform session expired, so the run's printed output is preserved verbatim in the artifact, whose single command regenerates both. The calibration analysis uses the earlier run's recorded predictions. The sign test counts ties as non-wins; this makes "hybrid wins half the time" easier to reject but does not change the count of wins. **External validity.** We study five benchmarks and one neural architecture family. A different detector might use structure better; our claim is that on these benchmarks no structure-dependent gain is visible and that the benchmarks cannot reveal one if it existed. **Adaptive attackers.** The artifact implements embedding-space PGD with expectation over the smoothing noise, GCG token search and graph evasion [47], [48], [49], [50], but none was run against the trained models, so we make no claim about adaptive robustness. Our results suggest that such attacks are hardly needed: held-out wordings already degrade every trained model.

What the results do not say. They do not say that learned detection of agent traffic is hopeless; the detector reaches a mean AUROC of 0.92 on AgentDojo attack families it never saw (0.78–0.99 per family) and the chunk scorer localises alarms to a tool or sentence. They say that current benchmarks cannot distinguish a detector that understands attacks from one that recognises templates, and that this should be measured before it is claimed.

7. Related Work

Pitfalls in security ML. Arp *et al.* [1] catalogue spurious correlations, sampling bias and inappropriate baselines; TESSERACT [2] shows that random splits overstate malware detectors. Our held-out folds and lexical control apply these lessons to agent protocols. **Benchmark critiques and injection detectors.** Closest to our message, Bhagwatkar *et al.* [3] show that a tool-input "minimiser" and a tool-output "sanitiser" reach full security on AgentDojo, Agent Security Bench, InjecAgent and τ-Bench, and attribute this to weak attacks, flawed success metrics and implementation bugs. They argue from the defence side, measuring attack success against agents. We argue from the detection side and supply a mechanism: grouped and held-out-family splits, a lexical control and a template audit show that learned detectors recognise templates, and that their false-alarm guarantees break across servers. PromptShield [4] and PIGuard [5] train deployable prompt-injection classifiers and study false alarms and over-defence on prompt corpora; they classify single prompts, whereas we evaluate on protocol episodes with held-out attack families. **Agent-protocol attacks and benchmarks.** MCP and A2A threats are surveyed in [6], [8]; MCPTox, MCP-SafetyBench, MCPSecBench, AgentDojo and InjecAgent [7], [9], [10], [14], [15] measure attack success against agents, not detection from traffic. **Monitors and design-level defences.** ClawGuard [12] enforces tool-access policies, AgentArmor [11] builds program-dependency graphs from traces, DataSentinel [19] detects prompt injection game-theoretically; CaMeL [16], MELON [17], Progent [18] and Task Shield [13] prevent injection outcomes by design and are complementary to detection. **Learned network intrusion detection.** Graph [38], Transformer [39] and state-space [33] detectors operate on flow features, not protocol message semantics. **Certifica-**

tion and calibration. Randomised smoothing [40], sparse smoothing for graphs [44], [45], [46], erase-and-check for prompts [43], Lipschitz-margin training [41], [42] and attention Lipschitz analysis [36], [37]; conformal prediction [28] and its adaptive variant [29]; calibration of neural networks [27].

8. Conclusion

We measured what learned detectors learn from LLM-agent protocol traffic on five benchmarks, with grouped splits, held-out-template folds, simple controls on identical episodes, five seeds, conformal alarm rules and certificates. The strongest neural detector we could build is matched or beaten by a bag-of-words classifier in distribution, a calibrated hybrid rarely improves on that classifier under shift, and no model generalises across attack families. A template audit explains why: a few dozen imperative n-grams carry most of MCPTox's signal, a single benign filler sentence separates InjecAgent, and detection is easiest exactly where test attacks resemble training attacks. Chunk-level payload scoring is the one detector design choice that clearly helps; distribution-free false-alarm guarantees fail when servers change, and adaptive conformal inference recovers them at a recall cost. For researchers, the result is a protocol — held-out families, lexical and length controls, enough seeds — that a detection claim should pass; for benchmark builders, a call for paraphrased attacks and large benign classes; for practitioners, a reminder that a detector's alarm rate must be monitored per server population. The artifact reruns everything on one GPU in under three hours.

References

[1] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Security*, 2022.

[2] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. USENIX Security*, 2019.

[3] R. Bhagwatkar *et al.*, "Indirect prompt injections: Are firewalls all you need, or stronger benchmarks?" arXiv:2510.05244, 2025.

[4] D. Jacob *et al.*, "PromptShield: Deployable detection for prompt injection attacks," in *Proc. ACM Conf. Data and Application Security and Privacy (CODASPY)*, 2025.

[5] H. Li, X. Liu, N. Zhang, C. Xiao, "PIGuard: Prompt injection guardrail via mitigating overdefense for free," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2025.

[6] X. Hou *et al.*, "Model Context Protocol: Landscape, security threats, and future research directions," arXiv:2503.23278, 2025.

[7] Y. Yang *et al.*, "MCPSecBench: A systematic security benchmark and playground for testing model context protocols," arXiv:2508.13220, 2025.

[8] A. Lotfi, M. M. Rahman, I. Karim, E. Bertino, "A2ABreak: Systematic security analysis of the A2A protocol," arXiv:2609.10871, 2026.

[9] T. Zong *et al.*, "MCP-SafetyBench: A benchmark for safety evaluation of LLMs with real-world MCP servers," ICLR 2026, arXiv:2512.15163.

[10] Z. Wang *et al.*, "MCPTox: A benchmark for tool poisoning attack on real-world MCP servers," AAAI 2026, arXiv:2508.14925.

[11] P. Wang *et al.*, "AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection," arXiv:2508.01249, 2025.

[12] W. Zhao *et al.*, "ClawGuard: Securing LLM agents through deterministic tool-access policies," arXiv:2604.11790, 2026.

[13] F. Jia *et al.*, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," arXiv:2412.16682, 2024.

[14] E. Debenedetti *et al.*, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," NeurIPS 2024.

[15] Q. Zhan *et al.*, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," Findings of ACL 2024.

[16] E. Debenedetti *et al.*, "Defeating prompt injections by design (CaMeL)," arXiv:2503.18813, 2025.

[17] K. Zhu *et al.*, "MELON: Provable defense against indirect prompt injection attacks in AI agents," ICML 2025, PMLR 267:80310–80329.

[18] T. Shi *et al.*, "Progent: Programmable privilege control for LLM agents," arXiv:2504.11703, 2025.

[19] Y. Liu *et al.*, "DataSentinel: A game-theoretic detection of prompt injection attacks," IEEE S&P 2025, arXiv:2504.11358.

[20] N. Reimers, I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," EMNLP 2019.

[21] W. Wang *et al.*, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained Transformers," NeurIPS 2020.

[22] T. G. Dietterich, R. H. Lathrop, T. Lozano-Pérez, "Solving the multiple instance problem with axis-parallel rectangles," *Artificial Intelligence*, vol. 89, 1997.

[23] M. Ilse, J. Tomczak, M. Welling, "Attention-based deep multiple instance learning," ICML 2018.

[24] K. Lee, K. Lee, H. Lee, J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," NeurIPS 2018.

[25] O. Ledoit, M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *J. Multivariate Analysis*, vol. 88, 2004.

[26] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, MIT Press, 1999.

[27] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On calibration of modern neural networks," ICML 2017.

[28] V. Vovk, A. Gammerman, G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.

[29] I. Gibbs, E. Candès, "Adaptive conformal inference under distribution shift," NeurIPS 2021.

[30] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian J. Statistics*, vol. 6, 1979.

[31] A. Gu, T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[32] A. Gu *et al.*, "On the parameterization and initialization of diagonal state space models," NeurIPS 2022.

[33] T. Wang *et al.*, "NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba," in *Proc. IEEE 32nd Int. Conf. Network Protocols (ICNP)*, 2024, pp. 1–11.

[34] S. Brody, U. Alon, E. Yahav, "How attentive are graph attention networks?," ICLR 2022.

[35] D. Xu *et al.*, "Inductive representation learning on temporal graphs," ICLR 2020.

[36] G. Dasoulas, K. Scaman, A. Virmaux, "Lipschitz normalization for self-attention layers with application to graph neural networks," ICML 2021.

[37] H. Kim, G. Papamakarios, A. Mnih, "The Lipschitz constant of self-attention," ICML 2021.

[38] W. W. Lo *et al.*, "E-GraphSAGE: A graph neural network based IDS for IoT," IEEE/IFIP NOMS 2022.

[39] Z. Wu *et al.*, "RTIDS: A robust transformer-based approach for IDS," IEEE Access, vol. 10, 2022.

[40] J. M. Cohen, E. Rosenfeld, J. Z. Kolter, "Certified adversarial robustness via randomized smoothing," ICML 2019.

[41] Y. Tsuzuku, I. Sato, M. Sugiyama, "Lipschitz-margin training," NeurIPS 2018.

[42] M. Fazlyab *et al.*, "Efficient and accurate estimation of Lipschitz constants for deep neural networks," NeurIPS 2019.

[43] A. Kumar *et al.*, "Certifying LLM safety against adversarial prompting," COLM 2024, arXiv:2309.02705.

[44] A. Bojchevski, J. Gasteiger, S. Günnemann, "Efficient robustness certificates for discrete data: Sparsity-aware randomized smoothing for graphs, images and more," ICML 2020.

[45] J. Schuchardt *et al.*, "Localized randomized smoothing for collective robustness certification," ICLR 2023.

[46] J. Schuchardt *et al.*, "Hierarchical randomized smoothing," NeurIPS 2023.

[47] M. Nasr *et al.*, "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," USENIX Security 2026, arXiv:2510.09023.

[48] F. Tramèr *et al.*, "On adaptive attacks to adversarial example defenses," NeurIPS 2020.

[49] A. Athalye, N. Carlini, D. Wagner, "Obfuscated gradients give a false sense of security," ICML 2018.

[50] A. Zou *et al.*, "Universal and transferable adversarial attacks on aligned language models (GCG)," arXiv:2307.15043, 2023.

Appendix A. Detector Equations

Selective SSM. For each agent a, the projected inputs $u_1^a, \dots, u_L^a$ pass through $S = 4$ blocks. In each block a continuous state-space model $h'(t) = Ah(t) + Bu(t)$, $y(t) = Ch(t)$ with state $h(t) \in \mathbb{R}^{d_s}$ ($d_s = 16$) is discretised with step Δ_t (zero-order hold): $\bar{A}_t = \exp(\Delta_t A)$, $\bar{B}_t = (\Delta_t A)^{-1}(\exp(\Delta_t A) - I)\Delta_t B_t$, and

$$h_t^a = \bar{A}_t\, h_{t-1}^a + \bar{B}_t\, u_t^a, \tag{4}$$

$$z_t^a = C_t\, h_t^a \odot \mathrm{SiLU}\big(W_g\, u_t^a\big), \tag{5}$$

where Δ_t, B_t, C_t depend on the input (the "selective" part [31]), $\odot$ is element-wise multiplication, W_g a learned gate matrix, and $\mathrm{SiLU}(u) = u\,\sigma(u)$. A is diagonal with negative real entries (S4D-Real [32]), so the memory decays. A depthwise causal convolution precedes (4) and each block is residual.

Temporal GATv2. For agents a, a' joined by an edge with timestamp t_e, the attention logit is $e_{a,a'} = q^\top \mathrm{LeakyReLU}(W_r[z^a \| z^{a'} \| \phi_{a,a'} \| \Phi(t - t_e)])$, where $\|$ is concatenation, $\phi_{a,a'}$ the edge features, $\Phi(\Delta t)$ the TGAT time encoding [35] with $d_T = 64$ learnable frequencies, and q, W_r, W_v learned; LipschitzNorm [36] rescales the logits, $H = 4$ heads are averaged.

TABLE 7. LIPSCHITZ BOUNDS VERSUS THE SPECTRAL CAP c (DEFAULT 3.68 M-PARAMETER CONFIGURATION, SYNTHETIC INPUTS). $\hat{L}_f$: LARGEST OUTPUT CHANGE PER UNIT INPUT CHANGE FOUND BY 200-STEP PROJECTED GRADIENT ASCENT, A LOWER BOUND ON THE TRUE CONSTANT (EVALUATED AT $c = 0.5$ ONLY).

c	L_{ssm}	L_{gat}	Block product	Residual (3)	$\hat{L}_f$
0.50	0.06	1.46	0.023	3.09	0.083
0.75	12.88	5.50	39.87	—	—
1.00	58.84	14.73	651.80	—	—
1.25	89.74	32.30	2482.52	—	—
2.00	177.96	67.99	10364.17	—	—

Appendix B. Lipschitz Analysis

Assumption 1 (Bounded operating regime)**.** *(i) Inputs satisfy* $\|u_t\|_2 \le X_{\max}$*; (ii)* $\Delta_t \in [\Delta_{\min}, \Delta_{\max}]$*; (iii) the projections producing* B_t*,* C_t*,* Δ_t *have Jacobian spectral norms at most* M_B, M_C, M_Δ*; (iv)* A *is real diagonal with* $\Delta_{\max} \max_i A_i < 0$*, so* $\|\bar{A}_t\|_2 \le \rho < 1$*.*

Lemma 1 (Selective block)**.** *Under Assumption 1, with* $\beta = \sup_t \|\bar{B}_t\|_2$*,* $\gamma_C = \sup_t \|C_t\|_2$*,* $\kappa_g = \|W_g\|_2$ *and* $L_\sigma \approx 1.0998$ *the Lipschitz constant of SiLU,* $L_{ssm} \le \gamma_C \beta (1 + L_\sigma \kappa_g)/(1-\rho) + \gamma_C \beta X_{\max}(M_B + M_C + M_\Delta)/(1-\rho)^2$*.*

Proof. Unrolling (4), $\|h_t(u) - h_t(u')\| \le \sum_{k \ge 0} \rho^k \beta \|u_{t-k} - u'_{t-k}\| \le \beta/(1-\rho)\, \|u - u'\|_\infty$. Multiplying by γ_C and applying the product rule $\|fg\|_{\text{Lip}} \le \|f\|_\infty L_g + \|g\|_\infty L_f$ to the gate gives the first term; the input-dependence of $\bar{B}_t, C_t, \Delta_t$ contributes the second. The convolution is folded into β through its Toeplitz norm. □

Lemma 2 (LipschitzNorm GATv2)**.** *With LipschitzNorm constant* η_L*, and under the full normalisation of [36] on the bounded domain of Assumption 1,* $L_{gat} \le \|W_v\|_2 (1 + \eta_L \|q\|_2 \|W_r\|_2)$*.*

Our implementation rescales the attention logits only, so Lemma 2 is not claimed for the trained models; Table 7 evaluates the bounds on synthetic inputs. The SiLU constant: $\mathrm{SiLU}'(u) = \sigma(u)(1 + u(1 - \sigma(u)))$ peaks at $u^* \approx 2.3994$ with value ≈ 1.0998. Table 7 shows the bounds as the spectral cap c on every linear layer grows.

Appendix C. Per-Fold Held-Out Results

Table 8 lists all 31 held-out folds with the template similarity of §4.2. Two patterns support the conclusions. First, *ranking survives where thresholds fail*: on A2A the lexical baseline ranks two held-out classes perfectly (AUROC 1.00 on `indefinite_interrupted_tasks` and `unbounded_delegation_depth`) yet reaches macro-F1 0.33 on the first, because a threshold tuned on the other ten classes does not transfer — the same failure that breaks split conformal in §4.4. Second, the

TABLE 8. ALL HELD-OUT FOLDS: MEDIAN SIMILARITY OF TEST ATTACKS TO THE NEAREST TRAINING ATTACK (SIM.), AND TEST MACRO-F1 / AUROC AT THE VAL_{SEL}-TUNED THRESHOLD AS PRINTED BY THE STUDY RUNNER (RAW SCORES; DETECTOR: MEAN OF SEEDS 0–2, A2A SEED 0; LEXICAL: ONE DETERMINISTIC FIT).

Benchmark	Held-out fold	Sim.	SSM-GAT	Lexical
A2A (class)	agent_card_jws_trust	0.41	.330/.338	.333/.449
	artifact_chunk_corruption	0.79	.333/.453	.333/.489
	cross_client_context_injection	0.36	.333/.756	.333/.622
	indefinite_interrupted_tasks	0.46	.333/.556	.333/1.00
	multi_hop_identity_loss	0.29	.333/.436	.333/.569
	post_revocation_stream	0.72	.444/.760	.433/.400
	scope_amplification	0.30	.597/.573	.597/.893
	stale_task_state	0.29	.333/.173	.333/.284
	unattested_skill_claims	0.41	.333/.587	.466/.347
	unbounded_delegation_depth	0.48	.466/.858	.864/1.00
	unverified_webhook_url	0.26	.280/.209	.641/.609
MCPTox	paradigm Template-1	0.18	.534/.646	.832/.950
	paradigm Template-2	0.23	.525/.678	.587/.862
	paradigm Template-3	0.20	.524/.641	.589/.851
	wrapper `ignore_previous`	0.19	.581/.662	.854/.849
	wrapper `important_tag`	0.21	.524/.704	.574/.843
	wrapper `plain`	0.21	.500/.682	.616/.866
MCP-SafetyBench	host/user	0.29	.420/.736	.498/.646
	name competition	0.43	.440/.460	.492/.671
	tool poisoning	0.40	.482/.486	.494/.661
	result injection	0.23	.485/.703	.628/.700
AgentDojo	`direct`	0.70	.618/.782	.871/.938
	denial of service	0.52	.806/.952	.949/.995
	`ignore_previous`	0.74	.856/.943	.804/.966
	`important_instructions`	0.64	.896/.985	.949/1.00
	`injecagent`	0.73	.907/.976	.899/.986
	`tool_knowledge`	0.70	.705/.854	.975/1.00
InjecAgent	attacker cases 0	0.33	1.00/1.00	1.00/1.00
	attacker cases 1	0.31	.965/1.00	1.00/1.00
	attacker cases 2	0.31	1.00/1.00	1.00/1.00
	attacker cases 3	0.35	.802/.996	.818/1.00

detector is *not* uniformly worse: it ranks better than lexical on A2A `cross_client_context_injection`, `post_revocation_stream` and `unattested_skill_claims`, and on MCP-SafetyBench host/user and result injection, all structural or placement-borne attacks, but none of these advantages is large enough, or consistent enough across seeds, to change the averages.

Appendix D. Additional Measurements

Paired comparison. Table 9 gives the hybrid-minus-lexical paired bootstrap behind §4.1. **Conformal alarms.** Table 10 gives test FPR and recall for all four models under both conformal rules, including the novelty score, whose low false-alarm rate comes with recall of 0.03–0.25. **Certificates.** Table 11 gives certified accuracy for all three noise levels.

TABLE 9. HYBRID MINUS LEXICAL, PAIRED BOOTSTRAP OVER TEST EPISODES (2,000 RESAMPLES, SHARED ACROSS THE FIVE SEEDS); p FOR MACRO-F1 IS HOLM-ADJUSTED ACROSS BENCHMARKS.

Benchmark	ΔF1 [95% CI]	p_{Holm}	ΔAUROC [95% CI]	p
A2A sandbox	$+.006$ $[-.005, +.016]$	1.00	$+.001$ $[-.000, +.003]$	.17
MCPTox	$+.001$ $[+.000, +.004]$	1.00	$-.001$ $[-.002, +.001]$	.25
MCP-SafetyBench	$+.025$ $[-.038, +.094]$	1.00	$+.020$ $[-.010, +.061]$	.26
AgentDojo	$-.040$ $[-.077, -.011]$	.020	$+.000$ $[+.000, +.000]$	1.00
InjecAgent	$+.000$ $[+.000, +.000]$	1.00	$+.000$ $[+.000, +.000]$	1.00

TABLE 10. CONFORMAL FALSE-ALARM CONTROL AT $\alpha = 0.05$: TEST FPR / RECALL, MEAN OVER FIVE SEEDS. "—": FEWER THAN 19 BENIGN CALIBRATION EPISODES (SPLIT CONFORMAL) OR FEWER THAN 5 (ACI).

Benchmark	Rule	SSM-GAT	Lexical	Novelty	Hybrid
A2A sandbox	split	.033 / .956	.048 / .958	.034 / .213	.043 / .964
	ACI	.050 / .847	.052 / .894	.048 / .253	.048 / .878
MCPTox	split	.171 / .506	.186 / .590	.029 / .035	.186 / .596
	ACI	.057 / .233	.071 / .221	.060 / .086	.071 / .223
MCP-SafetyBench	ACI	.058 / .400	.125 / .514	.092 / .211	.075 / .691
AgentDojo	ACI	.114 / .653	.143 / .433	.071 / .033	.114 / .591
InjecAgent	—	—	—	—	—

TABLE 11. CERTIFIED ACCURACY $\text{CertAcc}(r)$ AND ABSTENTION RATE (ABS.) OF THE SEED-0 FIVE-SEED MODELS; 100 TEST WINDOWS (59 FOR MCP-SAFETYBENCH), $n = 500$, $\alpha_{MC} = 10^{-3}$.

Benchmark	σ	Abs.	r=0	0.02	0.05	0.10	0.25
A2A sandbox	.05	.01	.95	.94	.90	.49	.00
	.10	.08	.87	.84	.67	.46	.00
	.25	.34	.55	.52	.39	.23	.03
MCPTox	.05	.02	.68	.65	.60	.42	.00
	.10	.05	.63	.60	.55	.40	.00
	.25	.19	.53	.48	.39	.24	.00
MCP-SafetyBench	.05	.05	.63	.49	.36	.10	.00
	.10	.34	.51	.39	.29	.10	.00
	.25	.54	.37	.29	.17	.07	.02
AgentDojo	.05	.03	.93	.93	.93	.76	.00
	.10	.02	.93	.93	.92	.71	.00
	.25	.09	.88	.87	.87	.68	.24
InjecAgent	.05	.02	.98	.98	.98	.96	.00
	.10	.02	.98	.98	.98	.96	.00
	.25	.03	.97	.96	.96	.96	.69

Appendix E. Hyperparameters and Compute

Detector: $d = 256$, SSM state $d_s = 16$, expansion 2, convolution width 4, $S = 4$ blocks; $H = 4$ attention heads, $d_T = 64$; chunk scorer $896 \to 128 \to 128$ (GELU), max+mean pooled; AdamW, learning rate 3×10^{-4}, weight decay 10^{-2}, cosine schedule with 200 warm-up steps, batch 32, class-balanced sampling, focal loss, $\lambda_L = 10^{-3}$, early stopping on val_{sel} macro-F1 (patience 5, at most 15/20/30/30/20 epochs for A2A/MCPTox/MCP-SafetyBench/AgentDojo/InjecAgent); fp32 on one T4. Five-seed study: main seeds 0–4, held-out seeds 0–2 (A2A 0), flat read-out, 2.8 GPU-hours. Earlier study: graph read-out, seeds 0–2, ablations 0–1, 7.1 GPU-hours. Payload encoder: chunks of ≤ 48 words or one per tool, at most 64 per

message, frozen `all-MiniLM-L6-v2`, 512 hash buckets. A2A sandbox: 200 attack/benign pairs per class (held-out folds: 100, at most 10 epochs). Novelty: PCA to ≤ 32 components, Ledoit–Wolf covariance. Hybrid: simplex grid step 0.1. ACI: $\gamma = 0.05$, random test order (seed 0). Bootstrap: 2,000 resamples. Smoothing: $n_0 = 32$, $n = 500$, $\alpha_{MC} = 10^{-3}$, $\sigma \in \{0.05, 0.10, 0.25\}$.

Appendix F. Protocol-Benchmark Canonicalisation

Loaders are in `apdet/data/protocol_traces/`; `reproducibility/PROTOCOL_BENCHMARKS.md` gives pinned source commits, the episode construction for each benchmark, expected counts and the shortcut baselines. Server source files are parsed statically and never executed. MCPTox and MCP-SafetyBench carry no license and are read from the reader's own clone, not redistributed. The sparse-smoothing graph-edit certificate [44] is implemented and unit-tested (on a 10-node graph with $p_+ = p_- = 0.05$, $n = 1000$, it certifies $(k_a, k_d) = (2, 2)$ edge additions/deletions), but was not evaluated at benchmark scale.

Appendix G. Ethics Considerations

All protocol traces are synthetic or from public benchmarks; no live third-party agent deployment was targeted and no human-subject data was collected. The adaptive-attack code could in principle be adapted to attack production MCP servers; we release the scaffolding needed to reproduce the paper but no working exploits against live services. Publishing that current benchmarks reward template recognition helps defenders more than attackers, who already know their templates.

Appendix H. Generative AI Usage

The benchmarks are public or generated deterministically from a fixed seed with no model in the loop (A2A sandbox); no closed model was queried to produce the data or the results. Generative AI was used for editing and for help with code; all outputs were checked by the authors.

Appendix I. Artifact Availability

An anonymised artifact is available at https://anonymous.4open.science/r/agent-protocol-detectors (identifier filled in at submission). It contains the benchmark loaders, the detector and its ablations, the lexical and novelty controls, the certificates, the adaptive attacks, the training and evaluation harness and the tests. `kaggle/run_kaggle_study.py --study all` reruns the five-seed study on one GPU in under three hours, and `scripts/analyse_v3.py` recomputes the hybrid, conformal, ACI and paired-test results from its saved scores. `reproducibility/kaggle_gpu_v3/` holds the five-seed run's printed results; `reproducibility/kaggle_gpu_v2/` the earlier study's runs, per-episode predictions, certificates and latency; `reproducibility/calibration_v2.json` the ECE analysis (`scripts/calibration_v2.py`); `reproducibility/template_audit.json` the template audit (`scripts/template_audit.py`, CPU only, about 20 minutes). The artifact is frozen at the submission deadline.